

A²S

Agente Autónomo Supremo con capacidades forenses

Informe de Análisis Integral ? A²S v1.6.0

250 criterios en 14 categorías · metodología: análisis estático (AST), métricas medidas, ejecución de la suite completa y verificaciones en vivo documentadas en el historial del proyecto.

Fecha: 2026-08-20 · Rama: arena/01a02086-a-2s · Licencia: MIT

Convención de veredictos por criterio: [SÍ] implementado y verificable · [PARCIAL] implementado con matices declarados · [NO] ausente (y por qué). Ningún criterio se maquilla: los NO son parte del resultado.

Resumen ejecutivo

A²S es un framework de agente autónomo en Python puro (stdlib, cero dependencias) que persigue objetivos con loops auto-optimizados hasta verificar su cumplimiento, y entrega informes con cadena de custodia criptográfica. La versión analizada (1.6.0) integra cuatro capas complementarias: (1) núcleo de recuperación fractal con escalera reintento->reparametrización->cambio de herramienta->división->replanificación; (2) SORL, un pool de proveedores legítimos con tres niveles de aprendizaje (cuota real observada, micro-ajuste de pesos, aptitud medida por tipo de tarea con puerta de incompetencia); (3) el Ciclo de Enriquecimiento, que aprende de repositorios públicos de GitHub hasta verificar capacidad; y (4) el nivel determinista (FSM + vigía por eventos) que resuelve lo predecible sin gastar un token y escala lo imprevisto al agente.

Su rasgo diferencial no es la potencia (un Auto-GPT con GPT-4 razona mejor) sino la confianza verificable: cada decisión deja huella auditable (ledger con hash chain, firmas HMAC, telemetría), cada límite está documentado como contrato (LIMITACIONES.md, 13 secciones) y las fronteras éticas están ejecutadas en el modelo de permisos, no prometidas en un documento. Ponderado sobre 250 criterios: 3,7/5, con máximo en documentación/ética (5,0) y mínimo en UI/UX (2,5).

Métricas medidas del código

Versión analizada	1.6.0 (rama arena/01a02086-a-2s, commits 58ca50f..03eab6f)
Líneas de código (a2s/)	6.489 en 24 módulos Python (stdlib al 100%)
Funciones / clases	309 funciones · 51 clases · CC media 4,8
Complejidad máx. (hotspots)	shell=33, execute_dag=31, evolve_step=26, _handler=23, execute_step=19
Pruebas	139 tests en 11 suites · ~3,4 s · 6 corridas consecutivas en verde
Dependencias externas	0 (ni runtime ni build más allá de setuptools)
Documentación	README 427 líneas · LIMITACIONES 439 líneas · 6 ejemplos ejecutables
Interfaces de usuario	CLI (14 comandos) · dashboard SSE · API Python · informes PDF/MD firmables
Persistencia	SQLite + JSONL append-only (hash chain) + JSON atómico + artefactos HMAC
Verificación en vivo	SORL: failover/429/aprendizaje · CE: API real GitHub · FSM: escalado nivel 0->1

Puntuaciones por categoría (0-5)

categoría	nota
C1 Arquitectura	4,0
C2 Funcionalidad	3,5
C3 UI/UX	2,5
C4 Analítica	3,5
C5 Crecimiento	3,0
C6 Seguridad	3,5
C7 Fiabilidad	3,5
C8 Código/Pruebas	3,5
C9 Documentación	5,0
C10 Ética	5,0
C11 Operación/Coste	3,5
C12 Escalabilidad	3,0
C13 Comparativa	3,5

Categoría 1 ? Arquitectura y Fundamentos Técnicos

1. Paradigma de programación

[PARCIAL] Principal: imperativo orientado a objetos con dataclasses (models.py, fsm.py); secundarios: funcional ligero (closures/comprendiones en scheduler y transports), declarativo (especificaciones JSON de FSM/watch/pool interpretadas por motores) y metaprogramación contenida (plugin_loader registra por etiquetas, sin RCE remoto).

2. Complejidad ciclomática

[Sí medida] 309 funciones, CC media 4,8 (sana; umbral clásico 10). Hotspots CC>19: tools.py:shell=33 (mini-shell con pipes/redirección/sustitución), provider_pool.py:execute_dag=31, planner.py:evolve_step=26, dashboard.py:_handler=23, loop.py:execute_step=19. Los cinco tienen tests dedicados pero son los candidatos a refactorizar.

3. Acoplamiento y cohesión

[PARCIAL] Coesión alta por módulo (una preocupación por archivo). Acoplamiento bajo y unidireccional (cli->loop->planner/tools; learner->provider_pool->providers); se evita la circularidad con imports perezosos (providers->provider_pool dentro de get_provider). Punto débil: cli.py conoce a todos (753 LOC, fachada creciente).

4. Patrones de diseño

[Sí] Strategy (BaseProvider: heurístico/OpenAI/pool; estrategias del scheduler), Factory (get_provider, build_pool_provider), Adapter (plugins externos como herramientas), Registry (ToolRegistry, plugin_loader), Observer (SSE del dashboard, on_event del watcher), Circuit Breaker (SORL), Retry con backoff (pool y learner), State (FSM), Pipe & Filters (execute_dag), Chain of Responsibility (escalera de recuperación), Facade (ProviderPool).

5. Antipatrones identificados

[Sí identificados] (a) God-function en cmd_run/cli y dashboard._handler; (b) duplicación de construcción de prompts LLM entre providers.py y provider_pool.py; (c) logging por print() en vez de módulo logging; (d) fascination: heurística que puntúa 'éxito' cualquier salida no vacía (LIMITACIONES §2 nº8-9 lo declara abiertamente). Ninguno bloqueante; los cuatro están en la lista de deuda.

6. Estrategia de gestión de estado

[Sí, híbrida] Mutable en ejecución (dataclasses Step/EndpointState), externalizada y persistente entre ejecuciones (SQLite episódico, strategies.json, governance.json, state.json del pool, fichas de conocimiento), e inmutable-auditable para la cadena de custodia (ledger JSONL append-only con hash chain). El estado nunca vive solo en RAM.

7. Modelo de concurrencia

[Sí] Hilos (ThreadPoolExecutor para fanout/DAG/fractales; locks RLock en pool y watcher), procesos (swarm: un worker por objetivo; sandbox de python_exec por subprocesso) y un hilo de servidor HTTP (webhook del watcher). No usa asyncio: con I/O de red acotado y stdlib-only, los hilos bastan y simplifican el razonamiento.

8. Eficiencia de algoritmos (Big O)

[Sí] Ventana deslizante de cuota $O(\text{rpm})$; orden topológico del DAG $O(V+E)$; utility del scheduler $O(1)$ por endpoint; ledger con caché de último hash: append $O(1)$ amortizado (bug $O(n^2)$ corregido en v1.1, 3,6× más rápido); verificación de cadena $O(n)$; emparejamiento de planes heurísticos $O(\text{plantillas} \times \text{palabras})$. Sin algoritmos superpolinómicos.

9. Estructura de datos subyacente

[Sí] Elegantes para su nicho: deque para ventanas de rate-limit, dicts como índices (herramientas, estados FSM, telemetría), listas de pasos con depends_on, JSONL append-only como log, SQLite para episodios. Idoneidad correcta; falta un índice invertido/emprendido para búsqueda semántica en la memoria (gap declarado).

10. Gestión de memoria

[Sí] Recolector de Python; gestión explícita y correcta de recursos externos (handle de telemetry.jsonl cerrado en close() con atexit resiliente; servidores con shutdown). Buffers acotados (lecturas HTTP limitadas a 60-200 KB; latencias con deque maxlen=200).

11. Stack tecnológico y justificación

[SÍ] Python >=3.9, 100% stdlib (urllib, sqlite3, http.server, threading, zipfile). Justificación estructural: LiveCD de ~500 KB ejecutable sin instalación, superficie de ataque de supply-chain nula y auditabilidad total del código. El coste asumido: reimplementar (mini-shell, cliente HTTP, motor PDF) en vez de depender.

12. Dependencias y su gestión

[SÍ: cero] No hay dependencias externas de ejecución (pyproject solo declara el paquete). Cero CVEs posibles de terceros; sin lockfile necesario. Las herramientas forenses externas (Sleuth Kit, Volatility...) son opcionales del operador, con lista blanca.

13. Compatibilidad y portabilidad

[PARCIAL] Python puro: portable a cualquier POSIX. Sesgo UNIX: /dev/shm para --ram, rlimits, nsjail/bwrap opcionales (degrada a nivel rlimits). Windows: el núcleo corre, pero sandbox avanzado y algunas herramientas shell no. Hardware: desde una Raspberry Pi (RAM de proceso ~decenas de MB); sin requisitos de GPU.

14. Interoperabilidad

[SÍ] Toda la frontera externa es HTTP+JSON: API compatible OpenAI (chat/completions) para cualquier LLM, REST de GitHub, webhooks entrantes, JSON exportable en cada comando (--json). No hay gRPC/protobuf ni SDKs de lenguajes: la interoperabilidad es por contrato REST simple, suficiente para el dominio.

15. Capacidad de refactorización

[PARCIAL] Red de seguridad: 139 tests (incluidos contratos del proveedor y regresiones de bugs históricos) + verificación criptográfica del ledger. Riesgo: sin medición de cobertura formal ni tests de mutación, los refactors de las 5 funciones CC>19 son delicados; recomendado extraer execute_dag y shell a módulos propios con tests primero.

16. Deuda técnica cuantificada

[SÍ] Cuantificada en este informe: CC media 4,8 con 5 funciones >19; ~90 LOC duplicados de prompts; cli.py en crecimiento (25+ flags); mensajes i18n hardcodeados; win-rate sin decaimiento (sesgo de popularidad); sin CHANGELOG formal. Estimación honesta: 2-3 jornadas de refactor puro para ponerlo todo por debajo de CC 15.

17. Resiliencia a fallos

[SÍ] Degradación en cascada demostrada en vivo: endpoint LLM caído -> cuarentena -> failover -> fallback heurístico -> misión cumplida igualmente. Circuit breaker, backoff exponencial, checkpoints reanudables, supervise (relanzamiento hasta éxito), sandbox que contiene fallos de memoria/CPU en python_exec.

18. Tolerancia a particiones (CAP)

[PARCIAL] Es un sistema de nodo único: no hay partición interna que tolerar (C fuerte por diseño). Ante particiones EXTERNAS (proveedores LLM/GitHub caídos o lentos), el pool prioriza disponibilidad con consistencia eventual de su telemetría: cada proceso decide con el último snapshot y lo reconcilia al cerrar. Trade-off documentado, no accidental.

19. Consistencia de datos

[SÍ] Fuerte y verificable dentro del workspace: ledger append-only con hash chain SHA-256 y detección de truncación (v1.1.1), firmas HMAC de informe y artefactos (a2s verify). Eventual solo en la capa de aprendizaje agregado del pool (snapshots por proceso).

20. Latencia end-to-end

[SÍ medida] Nivel 0 (FSM): milisegundos por transición, misión completa en <1 s. Nivel 1 heurístico: misión demo en segundos (6 iteraciones, ~0,1 s de ledger). Con LLM: dominada por el proveedor (p50 medidos en vivo: 23-251 ms según endpoint; hasta 120 s de timeout). El fanout paralelo convierte N llamadas en ~max(N/paralelismo) muros.

21. Rendimiento bajo carga

[PARCIAL] Verificado empíricamente hasta escala de demostración (12 subtarefas/8 hilos, enjambres de N procesos, 139 tests en 3,4 s). Sin benchmarks de estrés formales ni perfiles de CPU: la política de cuotas auto-impuesta del pool es el principal protector bajo carga real.

22. Consumo de recursos

[SÍ] CPU/RAM mínimos (proceso Python estándar; LiveCD ~500 KB en disco, workspace volátil en RAM opcional). Red: solo la que usan proveedor/GitHub con ventanas de cuota conservadoras. Disco: SQLite+JSONL de crecimiento lineal y acotado por maxlen en latencias (rotación del telemetry.jsonl no automática: gap menor).

23. Estrategia de persistencia

[SÍ, local-first] Cinco capas: SQLite (memoria episódica), JSONL append-only con hash chain (ledger), archivos JSON con escritura atómica os.replace (estrategias, gobernanza, state del pool, fichas de conocimiento), artefactos firmados en workspace, y snapshots temporales (telemetría). Todo dentro del workspace: copiar el workspace = migrar el sistema.

24. Introspección y depuración en runtime

[SÍ] a2s doctor (diagnóstico de entorno/sandbox/red), a2s verify (cripto), pool-status --json, traza de estados del FSM, verbose por ronda de planificación, dashboard SSE en vivo, ledger consultable por evento. Falta: tracing distribuido y profiler integrado (cProfile externo siempre disponible al ser Python).

25. Complejidad computacional teórica vs empírica

[PARCIAL] Las cotas teóricas están razonadas en el código (ver criterio 8); las empíricas existen pero dispersas: suite 139/3,4 s, learner 5,1 s con API real, latencias p50/p95 por endpoint en telemetría. No hay un banco de pruebas formal que una ambas caras (roadmap).

Categoría 2 ? Funcionalidades y Características

26. Alcance funcional vs especificación original

[Sí] La directiva original (ambiciosa y con peticiones ilegales) está mapeada 1:1 en `directiva.py`: cada capacidad pedida tiene implementación legítima equivalente o un NO explícito razonado (backdoors, minería, manipulación temporal). La tabla del README es el contrato de alcance y LIMITACIONES §1-13 su contrapartida crítica.

27. Completitud de features

[PARCIAL] Completas y verificadas en vivo: loop de recuperación fractal, SORL con tres niveles de aprendizaje, CE con API real de GitHub, FSM/watcher con escalado, firma HMAC, sandbox por capas, dashboard. Incompletas por diseño: búsqueda de código GitHub (solo README), provisionamiento spot, BOINC (diseño en §12).

28. Robustez ante entradas no esperadas

[Sí] Capas múltiples: `classify_forbidden` (30+ patrones de abuso), validación estática de especificaciones FSM antes de ejecutar, `_extract_json` tolerante a prosa, `validators` de esquema por kind en el pool, errores de acción convertidos en observaciones enrutables. Falta fuzzing sistemático (roadmap).

29. Extensibilidad

[Sí] Tres puntos de extensión limpios: implementar `BaseProvider` (nuevo motor de razonamiento), plugins con etiquetas cargados bajo demanda, endpoints del pool por configuración JSON. Añadir una herramienta = registrarla en `ToolRegistry`.

30. Modularidad funcional

[Sí] 24 módulos con fronteras claras por preocupación (loop, planner, memory, ledger, signing, auth, sandbox, providers, `provider_pool`, learner, fsm, dashboard...). La granularidad de paquete es plana (sin subpaquetes): aceptable a 6,5 k LOC.

31. Granularidad de funciones y servicios

[PARCIAL] Herramientas atómicas bien granuladas (`read_file`, `sha256sum`...); comandos CLI deliberadamente gruesos (una misión = un comando). El DAG y el fanout permiten elegir granularidad de subtareas en runtime.

32. Parametrización y configurabilidad

[Sí] Config con 25+ parámetros, flags CLI espejo, variables de entorno (`A2S_*`), `pool.json` (endpoints, pesos, estrategia), especificaciones FSM/watch JSON, límites adaptativos renovables por replanificación. Todo el comportamiento límite es del operador.

33. Automatización de procesos internos

[Sí] Replanificación automática, failover sin intervención, auto-aprendizaje de cuota real, micro-ajuste de pesos, puerta de incompetencia por medición, relanzamiento supervise, rotación de variantes ante estancamiento. El sistema se autogestiona dentro de los presupuestos fijados.

34. Capacidad de scripting del usuario

[Sí] Mini-shell del agente (`$VAR`, globs, `$(...)` con la misma política de permisos), `python_exec` aislado, y API Python importable (`ProviderPool`, `FSMEngine`, `Learner` son ciudadanos de primera: los ejemplos/ son scripts reales).

35. Flujos de trabajo soportados

[Sí] Lineal (run), fractal en paralelo (`run_fractal/swarm`), especulativo (N planes puntuados por la red de gobernanza), DAG con dependencias (`execute_dag`), dirigido por eventos (watch) y determinista (fsm). Componibles entre sí (watch puede escalar a run).

36. Gestión de identidades y accesos (IAM)

[PARCIAL] Autenticación de un rol: tokens JWT-HS256 con expiración para el dashboard (cookie `HttpOnly`), tokens por workspace. Sin RBAC ni multiusuario real: es una herramienta de operador único por diseño (gap si se despliega como servicio).

37. Auditoría y logging de acciones

[Sí, fortaleza] Ledger append-only con hash chain verificable (detecta modificación y truncación),

firmas HMAC por artefacto, telemetry.jsonl del pool, watch.jsonl del vigía, registro de acciones denegadas por el modelo de permisos. Cadena de custodia de nivel forense.

38. Seguridad a nivel funcional

[PARCIAL] Sin SQL dinámico (sqlite parametrizada), mini-shell sin shell=True (shlex + lista blanca), paths confinados al workspace, allowlist de red. El dashboard es básico y NO ha pasado una auditoría web formal (CSRF/ClickJacking no modelados): no exponerlo sin reverse proxy. Declarado, no oculto.

39. Funcionalidades asíncronas

[PARCIAL] Asíncronía por hilos (fanout, SSE, webhook) y procesos (swarm); no existe asyncio (decisión stdlib documentada). Para el I/O acotado del dominio es suficiente; un C10K de miles de webhooks simultáneos no es el caso de uso.

40. Funcionalidades en tiempo real

[SÍ] Dashboard con Server-Sent Events (misiones en vivo), watcher reactivo (file/webhook disparan en <1 s), métricas del pool en caliente. Sin WebSockets bidireccionales (no se necesitan: el flujo es de salida).

41. Procesamiento por lotes (batch)

[SÍ] fanout (map paralelo con cuotas), execute_dag (lotes con dependencias), swarm (lote de objetivos, un worker cada uno), learn (lote de ciclos de estudio). Demostrado en vivo: 12 subtareas sobre 3 endpoints respetando cuotas.

42. Internacionalización y localización

[NO] Mensajes en español de primera clase, hardcodeados (identidad deliberada del proyecto). Sin catálogos gettext ni locale switching. Para un agente de operador, aceptable; para producto global, deuda.

43. Accesibilidad (WCAG)

[NO formal] CLI: depende del terminal del usuario. Dashboard: HTML mínimo sin auditoría WCAG (contraste, roles ARIA, navegación por teclado no verificada). Si el dashboard se vuelve producto, esto pasa de gap a bloqueo.

44. Funcionalidades offline

[SÍ] Modo 100% offline verificado: núcleo heurístico determinista sin red ni claves (--no-network), sandbox bloquea red por shim. La sincronización 'online' (pool, CE) es aditiva: su aprendizaje persiste y se reutiliza cuando vuelve la red.

45. Importación/exportación de datos

[PARCIAL] Exportación sólida: informes MD+JSON firmados, pool-status --json, ledger consultable, fichas JSON. Importación limitada: especificaciones (pool/fsm/watch) y objetivos; sin conectores CSV/DB externos de propósito general.

46. Versionado de datos y rollback

[PARCIAL] El ledger es inmutable (append-only: la historia nunca se reescribe) y los snapshots usan escritura atómica con reemplazo. Rollback real = restaurar copia del workspace (documentado); sin migraciones versionadas de esquema (gap).

47. Integración con APIs de terceros

[SÍ] Cualquier endpoint compatible OpenAI (OpenAI, Groq, Gemini, GitHub Models, OpenRouter, Ollama local) con claves del operador y cuotas respetadas; REST de GitHub (búsqueda + README, Retry-After obedecido); búsqueda DuckDuckGo; fetch HTTP genérico con allowlist.

48. Capacidades de búsqueda

[PARCIAL] Búsqueda web (DDG), búsqueda en GitHub (repositorios), grep/list_dir locales del agente. Sin búsqueda semántica/vectorial en la memoria episódica ni faceteada: gap estructural declarado en §13.2/roadmap.

49. Funcionalidades de notificación

[NO] Notifica por consola, dashboard y archivos (informe, watch.jsonl). Sin email, push ni webhooks salientes. El webhook ENTRANTE existe; el saliente es roadmap trivial (postear el resultado del escalado).

50. Colaboración multiusuario

[NO] Modelo de operador único por workspace. El 'equipo' más parecido es el enjambre: réplicas aisladas con memoria propia por objetivo, sin shared-state colaborativo.

51. Gestión de conflictos colaborativos

[NO/NA] Al no haber escritura concurrente multiusuario, no hay conflictos de edición. La concurrencia interna se serializa por locks y el ledger ordena causalmente los eventos (single-writer por workspace).

52. Personalización de la experiencia

[PARCIAL] Personalizable por configuración (estrategias del pool, pesos, umbrales, plantillas de planes heurísticos editables como datos). Sin perfil por usuario ni preferencias persistentes de UI.

53. Capacidad de aprendizaje y adaptación (ML)

[Sí, honesto] MLP de gobernanza entrenado online (predice éxito de pasos), neuroevolución de topología y pesos (a2s evolve), selección de estrategias por win-rate, rpm aprendido por observación, aptitud medida por kind con puerta de incompetencia. Todo ligero y explicable; nada de deep learning pesado (por diseño stdlib).

54. Análisis y reporting

[Sí] Informe forense autónomo por misión (MD+JSON firmado con HMAC), render legible, pool-status con p50/p95/éxito/coste por endpoint, resumen ejecutivo del learn, traza de estados del FSM. Los informes son artefactos firmados: auditables.

55. Automatización de respuestas a eventos

[Sí] El watcher completo: interval con jitter / cambio de archivo / webhook -> máquina determinista -> si el evento es imprevisto, escala sola al agente que lo resuelve y queda todo en watch.jsonl. Verificado en vivo con los tres disparadores.

Categoría 3 ? Interfaz de Usuario y Experiencia (UI/UX)

56. Principios de diseño seguidos

[PARCIAL] A CLI se le aplican Nielsen por analogía: consistencia, visibilidad del estado (trazas por ronda), prevención de errores (confirmaciones implícitas vía lista blanca), ayuda integrada. No hay diseño formal guiado por heurísticas documentadas: la UI es instrumental para operadores técnicos.

57. Consistencia visual y de interacción

[SÍ] Convenciones estables en todos los comandos: prefijo [A²S] con marca de estado (logro/parcial/fallo/error), tablas alineadas en pool-status, mismas flags comunes en run/demo/learn/fsm/watch. El dashboard repite la identidad visual del CLI.

58. Jerarquía visual y claridad

[SÍ] En consola: títulos de ronda sangrados, tablas con cabecera y regla, traza de estados como flechas (leer -> clasificar -> registrar -> hecho). La información crítica (veredicto final firmado) siempre cierra la salida.

59. Affordance de elementos interactivos

[PARCIAL] --help por subcomando con descripciones accionables; flags con defaults documentados; mensajes que dicen qué hacer después ('exporta GROQ_API_KEY', 'usa --attempts para ampliar'). El dashboard tiene botones mínimos y funcionales.

60. Feedback al usuario

[SÍ] Inmediato y granular: cada paso de cada ronda se imprime con su veredicto, el pool anuncia cuarentenas y migraciones en vivo, el learn reporta fichas nuevas y brechas detectadas, el watcher marca cada evento con su resolución por nivel.

61. Minimización de carga cognitiva

[PARCIAL] Buen avance por defaults sensatos (auto, demo, doctor); pero la superficie total (25+ flags, 14 comandos) exige lectura del README. a2s map y el mapa de capacidades son los paliativos estructurales.

62. Eficiencia de usabilidad

[SÍ] Tareas clave en 1-2 comandos: misión (run objetivo), diagnóstico (doctor), pool (pool-status), vigilancia (watch spec). Los flujos largos (learn) son un solo comando con progreso por ciclo.

63. Curva de aprendizaje

[SÍ dual] Novato: demo + README con ejemplos copiables + doctor explicativo. Experto: API Python interna, especificaciones JSON, plugins. El escalón real está en entender el modelo mental (presupuestos adaptativos, escalada de recuperación): documentado con diagramas ASCII.

64. Diseño responsivo

[PARCIAL] Consola: el texto se envuelve al ancho del terminal (líneas truncadas con aviso). Dashboard: HTML simple sin media queries: funcional en móvil, no optimizado.

65. Uso del color

[PARCIAL] En consola se usa semántica de símbolos más que color (funciona también en b/n y lectores de terminal). El dashboard usa color con moderación (estado verde/ámbar). No hay paleta definida formalmente.

66. Tipografía y legibilidad

[SÍ] Consola monoespaciada (alineación perfecta de tablas); dashboard sans-serif sistema; informe PDF generado con Helvetica/Courier y jerarquía 17/12,5/10,5/9,3 pt.

67. Iconografía

[PARCIAL] Símbolos consistentes (logro, parcial, fallo, escala, engranaje, átomo) con significado fijo aprendible; riesgo de ambigüedad para nuevos usuarios o lectores de pantalla (los símbolos no tienen texto alternativo en consola: limitación del medio).

68. Gestión del espacio en blanco

[SÍ] Salidas aireadas: tablas con regla separadora, bloques separados por línea en blanco, informes con secciones. En PDF, márgenes de 52 pt y sangrías por nivel.

69. Patrones de navegación

[SÍ] CLI por subcomandos (verbos) con --help coherente; dashboard de una página con flujo lineal; informes con índice implícito por secciones. Sin navegación anidada confusa.

70. Diseño de formularios y tasa de error

[PARCIAL] El único formulario es el login del dashboard (token de un campo, baja tasa de error posible). La entrada principal es por argumentos: argparse valida tipos y choices, los errores son claros. Sin medición empírica de tasas.

71. Manejo de errores desde el usuario

[SÍ, fortaleza] Errores accionables, no stacktraces desnudos: 'revisa la clave/base_url', 'sin claves detectadas: exporta X o crea pool.json', especificación FSM inválida lista cada error con línea de contexto. El sistema nunca muere en silencio: degrada e informa.

72. Microinteracciones

[PARCIAL] En consola: las marcas de progreso por paso son la microinteracción. En dashboard: SSE actualiza en vivo. No hay animaciones (deliberado: sobriedad forense).

73. Estética emocional

[SÍ] Identidad reconocible: el tono 'supremo pero honesto' es coherente en README, mensajes y LIMITACIONES; la honestidad radical como rasgo de marca genera confianza real (contrato de expectativas: LIMITACIONES.md).

74. Personalización de la interfaz

[NO] Sin temas, sin layout configurable, sin preferencias de UI persistentes. La personalización real es arquitectónica (config/estrategias), no cosmética.

75. Accesibilidad de la interfaz

[NO formal] Contraste del dashboard no auditado, sin ARIA, símbolos unicode sin alternativa textual en consola. En CLI, la accesibilidad depende del emulador de terminal del usuario. Deuda declarada.

Categoría 4 ? Capacidad Analítica y de Resolución de Problemas

76. Abstracción del problema

[SÍ] Cuatro capas de abstracción estables: objetivo -> plan (estrategia+pasos) -> paso (criterios de éxito verificables) -> ToolCall (params JSON). El agente razona en objetivos y verifica en evidencia: la abstracción separa intención de ejecución.

77. Descomposición de problemas

[SÍ, núcleo del proyecto] División fractal recursiva (paso bloqueado -> sub-agentes), DAG por olas topológicas, fanout map, enjambre por objetivos. Cada subtask hereda presupuesto fraccionado (subagent_share) y verificación propia.

78. Reconocimiento de patrones

[PARCIAL] Emparejado por palabras clave (plantillas heurísticas), regex objetivas (FSM), medición de aptitud por kind (patrón estadístico de fallos de esquema). Sin reconocimiento estadístico profundo de patrones en datos (no es un sistema de ML general: el MLP de gobernanza predice éxito, no descubre patrones).

79. Razonamiento deductivo, inductivo y abductivo

[PARCIAL] Deductivo: verificación contra success_criteria (el veredicto se deriva de la evidencia). Inductivo: win-rates, p50, rpm aprendido (generaliza de lo observado). Abductivo: solo con LLM externo (generar la mejor explicación del fallo); el núcleo heurístico no hipotetiza: por eso su evaluación es débil (§2 n°9).

80. Pensamiento crítico y alternativas

[SÍ] Planificación especulativa: N planes variantes compiten y la red de gobernanza elige; reparametrización con obligación de NO repetir el enfoque fallido; escalera de recuperación que agota una vía antes de cambiar de estrategia.

81. Pensamiento sistémico

[SÍ] El pool modela el sistema completo de recursos (latencias, cuotas, coste, aptitud, fallos) y optimiza globalmente, no por llamada; LIMITACIONES documenta las interacciones (p. ej. 'si tu FSM escala cada minuto, el coste 0 se evapora').

82. Heurística aplicada

[SÍ] Biblioteca de estrategias con win-rate, ventana de estancamiento, función de utilidad multi-objetivo ponderada, brechas por identificadores técnicos, resumen extractivo. Heurísticas explicables y parametrizables, no cajas negras.

83. Algorítmica del pensamiento

[SÍ] Todo lo que decide está en términos algorítmicos verificables: orden topológico, backoff exponencial, ventana deslizante, suavizado bayesiano (3 pseudo-muestras), selección por argmax de utilidad. Nada de 'magia': cada decisión tiene código y test.

84. Síntesis de información dispersa

[PARCIAL] goal_check sintetiza el summary de la misión; aggregate del DAG combina resultados parciales; fichas CE destilan READMEs. La síntesis de calidad (resumen coherente) requiere LLM: el modo stdlib es extractivo (pobre pero honesto, §11.2).

85. Análisis de causa raíz (RCA)

[PARCIAL] El escalado FSM reporta exactamente 'qué transición faltaba' (RCA de especificación); el ledger permite reconstruir causalmente cualquier fallo; el informe post-mortem guarda estado exacto reanudable. No hay RCA automático de fallos complejos (el operador analiza con verify/doctor).

86. Modelado matemático

[SÍ] Función de utilidad lineal ponderada con restricciones; MLP con sigmoide y entrenamiento online por error; fitness de neuroevolución sobre holdout; estimadores de cuota (80% de lo observado en vuelo) y de coste (tokens×tarifa por tier).

87. Simulación de escenarios

[SÍ] Planificación especulativa: se generan N futuros (planes variantes), se puntúan con la red entrenada en episodios reales y se ejecuta el mejor. Es simulación de banda con validación empírica

posterior (el win-rate real retroalimenta).

88. Optimización de procesos y recursos

[SÍ] Scheduler multi-objetivo (coste/velocidad/fiabilidad/aptitud ? riesgo de cuota), rpm aprendido con homeostasis, free-first con pago solo para lo que solo el pago sabe hacer (demostrado: 6/6 planes válidos al endpoint competente, gratis para el resto).

89. Toma de decisiones bajo incertidumbre

[SÍ] Verificación por consenso ponderado de 4 fuentes (misión autoritativa, proveedor, red neuronal, evidencia de progreso); decisiones del pool con telemetría parcial (priors suavizados); presupuestos renovables que evitan apostar todo a un intento.

90. Gestión de la ambigüedad

[PARCIAL] El evaluador distingue failed (reparametrizable) vs blocked (cambiar de estrategia): taxonomía explícita de ambigüedad. Pero objetivos vagos sin verificador de misión producen falsos 'cumplido' (LIMITACIONES §2 nº9: advertencia permanente).

91. Resolución de conflictos lógicos y de datos

[SÍ] verify del ledger detecta contradicciones (truncación, firma incoherente); circuito con media apertura resuelve el conflicto '¿caído o saturado?'; escritura atómica con os.replace elimina lecturas a medias; single-writer por workspace.

92. Trazabilidad de decisiones

[SÍ, fortaleza] Cada decisión deja huella verificable: pool_provider en cada plan LLM, traza de estados FSM, telemetría por llamada, ledger encadenado con HMAC, fichas con fuente y licencia. Se puede auditar por qué se tomó CUALQUIER decisión.

93. Evaluación de trade-offs

[SÍ] Los trade-offs estructurales están escritos y razonados, no implícitos: stdlib vs ecosistema, LiveCD vs ML pesado, FSM barata vs inflexible, coste vs aptitud, esperar Retry-After vs migrar. El informe que lee es en sí un ejercicio de trade-offs.

94. Pensamiento lateral y creatividad

[PARCIAL] Reinterpretación de la directiva (cada petición imposible -> equivalente legítimo) es pensamiento lateral institucionalizado; rotación de variantes y cambio de herramienta ante estancamiento. La creatividad genuina (soluciones nuevas) llega con LLM o con el operador: el núcleo es deliberadamente conservador.

95. Depuración sistemática

[SÍ] doctor (entorno), verify (cripto), pool-check (salud por endpoint), verbose por ronda, errores como observaciones enrutables en FSM, tests de regresión por cada bug corregido (la conversión '(acción sin salida)' corrupta, la brecha que mezclaba 'Error module named'...).

96. Análisis de precedentes

[SÍ] El mapa de directiva referencia 53 repositorios del estado del arte clasificados (§9 de LIMITACIONES); el CE busca y asimila precedentes reales de GitHub para problemas nuevos; los planes heurísticos son precedentes codificados.

97. Formulación y validación de hipótesis

[SÍ] Hipótesis = plan/variante; validación = verificador de objetivo + criterios por paso; puntuación previa por red de gobernanza (hipótesis sobre la hipótesis); el win-rate posterior cierra el ciclo. Falsacionista por diseño: sin verificación positiva, nada está 'cumplido'.

98. Reconocimiento de sesgos

[PARCIAL] Sesgos estructurales mitigados: 429s excluidos de la tasa de éxito (no culpar al saturado), prior bayesiano contra el pánico de 1 muestra, confianza reportada como evidencia (no sensación). Sesgos restantes declarados: win-rate sin decaimiento (popularidad histórica), selección CE por estrellas (popularidad).

99. Gestión de la complejidad

[SÍ] Estratificada: nivel 0 determinista para lo predecible, nivel 1 agente para lo imprevisto, CE para lo desconocido. Cada capa solo despierta a la siguiente cuando hace falta: la complejidad se paga solo cuando se usa (y el informe de escalado dice exactamente qué faltó).

100. Adaptación de estrategias en marcha

[Sí] La escalera completa: reintento -> reparametrización -> cambio de herramienta -> división fractal
-> detección de estancamiento (ventana de fallos) -> replanificación con variante distinta. A nivel
pool: cuarentena, circuito, rpm aprendido, pesos.

Categoría 5 ? Capacidad de Investigación y Crecimiento Autónomo

101. Autonomía en definir objetivos de investigación

[PARCIAL] El CE formula solo sus consultas de brecha a partir de fallos reales; supervise/goals persiguen objetivos sin intervención. Pero el objetivo MACRO siempre lo fija el operador: no hay 'curiosidad' autónoma (elección de qué investigar sin misión): correcto para una herramienta, límite real de 'agente supremo'.

102. Búsqueda y recuperación de información

[SÍ] GitHub Search API (repositorios por relevancia/estrellas, READMEs crudos), búsqueda web DDG, fetch HTTP genérico con allowlist, listado/lectura local. Con presupuesto (60 llamadas/sesión) y ventanas de cuota bajo el límite real.

103. Evaluación crítica de fuentes

[PARCIAL] Señales usadas: estrellas (proxy de calidad, débil: documentado el caso de un repo de 0 estrellas para consulta rara), licencia (registrada siempre), win-rate propio de la ficha tras uso. Sin evaluación de autoría, frescura (updated_at disponible, no ponderado) ni contraste cruzado de fuentes.

104. Síntesis de estado del arte

[PARCIAL] Con pool LLM: resúmenes y recetas por repo, fusionables en contexto de planificación (fichas top-4 por win-rate). Sin LLM: extractivo pobre. No genera 'informe de estado del arte' formal comparativo: es síntesis operativa, no académica.

105. Generación de nuevas hipótesis de investigación

[PARCIAL] Las brechas detectadas son hipótesis de conocimiento faltante; la planificación especulativa genera hipótesis de acción. No genera hipótesis científicas de nuevo cuño: eso requiere LLM y verificación externa (honestamente fuera de alcance stdlib).

106. Diseño de experimentos

[PARCIAL] El sistema experimenta constantemente (variantes de plan compiten, endpoints se prueban y miden) pero no DISEÑA experimentos controlados (sin A/B formal con hipótesis nula, sin tamaños de muestra pre-fijados). Es experimentación adaptativa, no metodología experimental.

107. Capacidad de meta-análisis

[PARCIAL] Agrega telemetría histórica (p50/p95, win-rates, tasas de éxito por kind) y la usa para decidir; el dashboard/estado dan la vista global. No hay meta-análisis estadístico formal (intervalos de confianza, efectos de tamaño).

108. Aprendizaje continuo

[SÍ] Persiste y reaprovecha entre ejecuciones: estrategias (win-rate), pesos de gobernanza (MLP), rpm efectivo, pesos del scheduler, aptitudes medidas, fichas de conocimiento. Cada ejecución arranca más sabia que la anterior (verificado en vivo: 429s -> aprende -> 0 429s en la tercera ronda).

109. Des-aprendizaje (unlearning)

[PARCIAL] Existe degradación controlada: recuperación gradual del rpm (+1 tras 20 éxitos limpios), ventana de fallos que jubila estrategias en desuso. NO existe caducidad de fichas de conocimiento ni decaimiento de win-rates: el conocimiento obsoleto persiste hasta borrarlo a mano (gap declarado).

110. Curación de lo aprendido (añadido)

[PARCIAL] Las fichas llevan win-rate atribuido y el contexto prioriza las ganadoras; el contenido prohibido se rechaza en la puerta. Pero no hay poda automática de fichas perdedoras ni detección de contradicciones entre fichas.

111. Memorización selectiva (añadido)

[SÍ] Solo persiste lo que demostró utilidad o es estructura del sistema; las latencias tienen deque maxlen=200 (memoria reciente), el ledger es la memoria permanente inmutable. Jerarquía de memoria real: corto plazo (contexto comprimido), medio (episodios SQLite), largo (estrategias/fichas).

112. Transferencia entre dominios (añadido)

[PARCIAL] Las plantillas heurísticas genéricas (explorar/documentar/verificar) transfieren a cualquier objetivo; los planes LLM transfieren patrones del corpus del modelo. Sin mecanismo explícito de

transferencia entre misiones A²S distintas (cada workspace aprende por su cuenta).

113. Autoevaluación calibrada (añadido)

[SÍ] La 'confianza' del learn es evidencia contable (ciclos, fichas aplicadas, victorias), nunca probabilidad subjetiva; el evaluador separa éxito/bloqueo/fallo. Calibración honesta: el sistema no afirma lo que no verificó.

114. Presupuesto de exploración (añadido)

[SÍ] Toda exploración tiene techo: max_cycles del learn, 60 llamadas de API, presupuestos renovables por replanificación, límite duro de tiempo real. Explorar infinito está prohibido por diseño (parada honesta con informe).

115. Detección de novedad (añadido)

[SÍ] El imprevisto es un ciudadano de primera clase: transición sin match -> escalado con evidencia; JSON con esquema inesperado -> aptitud a la baja; brecha de conocimiento -> consulta. El sistema SABE cuándo no sabe (y lo dice).

116. Archivo de fuentes con licencia (añadido)

[SÍ] Toda ficha guarda repo, URL, licencia SPDX, extracto citado y fecha: el conocimiento es atribuible y auditable. La licencia se registra ANTES de usar la idea (higiene legal mínima para un agente que aprende de código ajeno público).

117. Citabilidad de decisiones (añadido)

[SÍ] Un plan LLM cita su endpoint (pool_provider); una acción FSM cita su traza; un aprendizaje cita su ficha; un informe cita su firma HMAC. Nada se afirma sin pedigree.

118. Reproducibilidad (añadido)

[PARCIAL] El núcleo heurístico es determinista puro (misma entrada -> mismo plan); los tests lo fijan. El pool/fanout/jitter usan aleatoriedad no sembrada: carreras de hilos pueden variar qué endpoint sirve cada subtarea (los AGREGADOS son estables). Falta una flag --seed global.

119. Apertura y auditabilidad del código (añadido)

[SÍ] MIT, cero dependencias (todo el comportamiento está en 6,5 k LOC legibles), sin binarios, sin telemetría saliente propia. Un auditor puede leer TODO el sistema en una tarde: rara vez cierto en agentes actuales.

120. Auditoría del propio aprendizaje (añadido)

[SÍ] telemetry.jsonl registra cada llamada (éxito, latencia, 429, coste estimado); el state.json versiona rpm/pesos/aptitudes aprendidos; watch.jsonl registra cada escalado. El aprendizaje es un proceso auditable, no una caja negra.

121. Límites del aprendizaje reconocidos (añadido)

[SÍ] §10-11 declaran qué NO aprende: calidad semántica profunda (un plan válido pero estúpido puntúa 1,0), formatos de fuente cambiantes (la FSM no se auto-corrige), decaimiento de conocimiento. Saber qué no se sabe es parte del diseño.

122. Curiosidad segura (añadido)

[SÍ] Toda exploración externa pasa el mismo modelo de permisos (classify_forbidden sobre el contenido asimilado: un README que describa conductas prohibidas se rechaza y se registra). El agente puede leer mucho, pero no aprender mal.

123. Economía del aprendizaje (añadido)

[SÍ] Aprender cuesta: se mide (coste estimado por llamada en telemetry, \$ por endpoint) y se minimiza (free-first, extractivo sin LLM como modo pobre, reuso de fichas entre ejecuciones: pagar una vez, aprovechar para siempre).

124. Escalado del conocimiento al equipo (añadido)

[NO] Las fichas viven en un workspace: no hay repositorio compartido de conocimiento entre instancias/operadores (exportable a mano: son JSON). Roadmap: directorio compartido de fichas firmadas.

125. Independencia de proveedor para crecer (añadido)

[SÍ] El aprendizaje funciona en dos extremos: 100% offline (extractivo heurístico, coste 0) o con cualquier LLM OpenAI-compatible (resúmenes de calidad). El crecimiento no está rehén de un proveedor:

cambiar de pool cambia la calidad, no la capacidad.

Categoría 6 (añadida) ? Seguridad y Modelo de Permisos

126. Modelo de permisos de objetivos

[Sí] classify_forbidden con 30+ patrones (exfiltración, malware, evasión, phishing, cracking...) rechaza objetivos y acciones antes de ejecutar; lo denegado queda registrado en el ledger. Es un filtro de intención, no solo de sintaxis.

127. Confinamiento de shell

[Sí] Mini-shell propio: shlex + lista blanca de binarios (sin shell=True), globs y \$() re-expanden bajo LA MISMA política, redirecciones acotadas, --unsafe explícito bajo responsabilidad del operador.

128. Confinamiento de red

[Sí] --no-network global, allowlist de hosts (--allow-host repetible) aplicada a fetch/búsqueda, sandbox de python_exec que bloquea red por shim, webhook del watcher ligado a 127.0.0.1 por defecto.

129. Sandbox de ejecución por capas

[Sí] nsjail > bwrap > rlimits con degradación informada (doctor reporta el nivel real): RAM/CPU/procesos/fds limitados, filesystem no aislado en el nivel rlimits (limitación documentada \$5), paths confinados al workspace en herramientas.

130. Integridad criptográfica

[Sí] Ledger append-only con hash chain SHA-256 (detecta modificación y truncación por cruce con índice SQLite), HMAC-SHA256 de informe y artefactos con secreto por workspace, a2s verify valida todo el conjunto.

131. Autenticación de servicios propios

[Sí] Dashboard: localhost por defecto, --public con advertencia, tokens JWT-HS256 con expiración (a2s token) y cookie HttpOnly. Webhook entrante: sin auth (documentado: detrás de reverse proxy si se expone).

132. Gestión de secretos

[PARCIAL] Las claves de APIs viven en variables de entorno o pool.json con expansión \${VAR} (nunca se imprimen: pool-status muestra modelos, no claves). El secreto HMAC del workspace es local: compromiso del disco = compromiso de firmas (declarado).

133. Superficie de supply-chain

[Sí, excepcional] Cero dependencias de terceros: ningún pip install, ningún CVE heredado, lockfile innecesario. El CE además NUNCA ejecuta código de los repos que estudia (solo lee texto): riesgo de dependencia maliciosa por aprendizaje = 0.

134. Inyección (SQL/comando/HTML)

[Sí] SQL siempre parametrizada; comandos vía argv sin shell del sistema; el dashboard escapa lo esencial pero NO tiene auditoría XSS formal (gap si se expone a terceros); el motor PDF propio escapa paréntesis/backslash.

135. Resistencia a prompt-injection desde fuentes

[PARCIAL] El contenido asimilado pasa classify_forbidden; el contexto inyectado marca la frontera ('nunca ejecute código de las fuentes'). Pero un README malicioso podría influir prompts LLM downstream (sin aislamiento semántico de instrucciones): riesgo real reconocido, mitigado por permisos de ejecución, no eliminado.

136. Principio de mínimo privilegio por herramienta

[Sí] Cada herramienta declara network/destructive en su registro; read/write confinados al workspace; plugins limitados (max_plugins) y cargados bajo demanda solo si la misión los necesita.

137. Protección de denial-of-service propio

[Sí] Límites duros en todas las fronteras: timeouts por petición, presupuesto de llamadas de API, presupuesto de ciclos, buffers de lectura acotados (60-200 KB), ventanas de rpm auto-impuestas. El sistema no puede consumirse a sí mismo.

138. Trazabilidad de acciones de seguridad

[Sí] Cada denegación y cada verificación criptográfica queda en el ledger; el modelo de permisos es

auditable post-mortem (qué se intentó, qué se rechazó, por qué patrón).

139. Seguridad del LiveCD

[PARCIAL] El zipapp es el mismo código sin instalación (auditable); --ram opera en /dev/shm (nada en disco, pero también sin persistencia de secretos). Sin firma del artefacto zipapp: verificar checksum al distribuir (recomendación).

140. Criptografía usada

[PARCIAL] Primitivas correctas y stdlib: SHA-256, HMAC-SHA256, JWT-HS256. Sin TLS propio en dashboard/webhook (localhost o reverse proxy del operador), sin crypto-defensa avanzada (padding, KDF): proporcional al dominio de herramienta local.

141. Respuesta a incidentes del propio sistema

[PARCIAL] El ledger + informes firmados son la evidencia forense de lo que el propio sistema hizo (puede auditarse a sí mismo). No hay runbook de respuesta ni revocación de tokens emitidos (gap operativo).

Categoría 7 (añadida) ? Fiabilidad Operativa y Observabilidad

142. Degradación gradual ante fallos

[Sí, verificada en vivo] Cadena completa: endpoint enfermo -> cuarentena -> failover -> circuito abierto -> fallback heurístico -> misión continúa. Demo real: TLS bloqueado del sandbox -> 3 fallos -> degradación -> OBJETIVO CUMPLIDO igualmente.

143. Circuit breaker y backoff

[Sí] 3 fallos consecutivos abren circuito 30 s (duplicando hasta 300 s) con media apertura para sondas; backoff exponencial 5 s->300 s en saturaciones; Retry-After del servidor siempre obedecido (espera acotada + 1 reintento + parada honesta).

144. Checkpoints y reanudación

[Sí] Estado persistido por episodio/estrategia/gobernanza; --resume reanuda sobre workspace conservado; informes post-mortem con estado exacto y cadena de custodia para reanudar cualquier misión interrumpida.

145. Watchdog y auto-relanzamiento

[Sí] supervise reintenta la misión hasta verificación (con sueños configurables); watcher para por inactividad en vez de colgarse; atexit persiste aprendizaje aunque maten el proceso.

146. Idempotencia de re-ejecuciones

[PARCIAL] Misiones y máquinas pueden re-correser sin dañar estado (ledger solo añade; fichas no duplican repos); write_file sobrescribe. No hay transacciones ATÓMICAS multi-archivo: un fallo a mitad de misión deja artifacts parciales (auditable, pero no rollback automático).

147. Pruebas de caos aplicadas

[Sí, informales] Los tests inyectan 429/503/TLS-roto/JSON-basura/saturación y verifican migración y degradación; las demos en vivo bloquearon proveedores reales y el sistema respondió según diseño. Sin caos automatizado en CI (no hay CI: gap).

148. Copias de seguridad

[PARCIAL] Todo el estado es el workspace (copiable de un golpe); el ledger es la historia completa. Sin backup automático programado ni retención: política del operador.

149. Health checks

[Sí] doctor (entorno completo), pool-check (1 petición real por endpoint, mide latencia y valida claves), verify (cripto), validate de especificaciones FSM antes de operar.

150. Métricas operativas

[Sí] Telemetría por endpoint (total/ok/429/errores/p50/p95/tokens/coste), contadores del servidor mock, resumen por ejecución, watch.jsonl. Formato JSONL: exportable a Grafana externo (documentado).

151. Alertamiento

[NO] Detecta y registra todo, pero no alerta proactivamente (sin email/push/webhook saliente). El operador mira dashboard/status. Gap barato de cerrar (roadmap).

152. SLOs y error budgets

[NO] No hay SLO formales (no es un servicio multiusuario). Los equivalentes contractuales son los presupuestos: ciclos, tiempo, llamadas API: siempre acotados y reportados.

153. Recuperación ante desastre

[PARCIAL] Workspace = todo el estado (fácil de restaurar); LiveCD regenera el sistema en segundos sin instalación. Un desastre del workspace sin backup = pérdida total del aprendizaje acumulado (declarado).

154. Estabilidad de arranque y ciclo de vida

[Sí] Arranque <1 s (sin dependencias que cargar); shutdown limpio de servidores/handles; procesos hijos cancelados por deadline del padre (bug corregido v1.1 n°3).

155. Gestión de bloqueos y carreras

[Sí] Locks RLock simples y cortos, single-writer por archivo, escrituras atómicas (tmp+os.replace), ThreadPool con tope, colas con timeout. Tests específicos de carrera (12 subtareas/4-8 hilos) pasando

estable en 6 corridas consecutivas.

Categoría 8 (añadida) ? Calidad del Código y Pruebas

156. Cobertura de pruebas real

[PARCIAL] 139 tests en 11 suites, ~3,4 s, todos en verde (6 corridas consecutivas estables). Cubren contrato y regresión de cada módulo nuevo (pool: 41, fsm: 17, learner: 16). Sin medición de cobertura por línea (sin herramienta): la cifra real de líneas cubiertas es desconocida (gap honesto).

157. Pirámide de tests

[SÍ] Unidades puras (RateWindow, jitter, heurísticas), integración con fakes inyectados (transports HTTP falsos, registros reales), sistema end-to-end (misiones completas, demos en vivo contra servidores reales). Sin e2e de navegador.

158. Tests de contrato

[SÍ] BaseProvider es un contrato testeado (heurístico/OpenAI/pool intercambiables); el agregador del DAG tiene contrato documentado (y el demo lo rompió: se corrigió el código para cumplir el contrato, con test).

159. Tests de regresión por bug

[SÍ, disciplina] Cada bug corregido dejó test: truncación del ledger, $O(n^2)$ del append, plazo del padre, observaciones vacías del FSM, brecha que mezclaba texto de error, atribución de aptitud en prosa.

160. Determinismo de la suite

[SÍ] Las carreras de hilos se controlaron haciendo deterministas los casos sensibles (max_parallel=1 en aprendizaje de rpm); los esperas se inyectan (sleep_fn). Suite completa: 3,4 s constantes.

161. Tipado estático

[PARCIAL] Type hints en casi todas las firmas modernas (from __future__ import annotations), dataclasses fuertes. Sin mypy/pyright en verificación (habría que añadirlos como dev-deps: decisión pendiente).

162. Estilo y consistencia

[SÍ] Convención única (español consistente, docstrings ricos por módulo/función pública, ~100 columnas); sin linter/formatter formal (ruff/black: recomendación de adopción como dev-only sin romper stdlib runtime).

163. Gestión de hotspots de complejidad

[PARCIAL] Identificados y medidos (5 funciones CC>19, criterio 2); 4 de ellas con tests densos que permiten refactor seguro; falta el refactor mismo (deuda cuantificada).

164. Duplicación

[PARCIAL] Baja en lógica (los motores se reutilizan: pool sirve a learner, FSM usa ToolRegistry), media en prompts LLM (providers vs pool) y en parsers de headers (retry-after aparece 2 veces con semántica distinta: servidor vs proveedor).

165. Nombrado y lenguaje

[SÍ] Español consistente de dominio (objetivo, plan, escalera, cuarentena, brecha, ficha) que hace el código autoexplicativo para su público; inglés solo en identificadores técnicos estándar.

166. Documentación en el código

[SÍ] Cada módulo abre con un docstring-manifiesto (qué hace, fronteras, referencias a LIMITACIONES); funciones públicas con docstrings con matices honestos ('pobre pero honesto', 'no evita', 'respeta').

167. Tests de seguridad

[SÍ] Suite test_hardening específica (tokens, sandbox, allowlist, denegaciones); tests de rechazo de contenido prohibido en learner y FSM; verificación criptográfica testeada (truncación detectada).

168. Mutación y fuzzing

[NO] Sin tests de mutación ni fuzzing sistemático de entradas (las validaciones existen pero no se torturan con mutantes aleatorios). Roadmap barato: fuzz de especificaciones FSM y de JSONs externos.

169. CI/CD

[NO] No hay pipeline (la suite corre en local/manual). Con cero dependencias, un GitHub Actions de 6

líneas ejecutaría los 139 tests: mejora recomendada nº1 de proceso.

Categoría 9 (añadida) ? Documentación y Transparencia

170. Documentación de usuario

[SÍ] README de 427 líneas con tabla capacidad->implementación real, ejemplos copiables de cada comando, sección ética y arquitectura con diagramas ASCII.

171. Documentación de límites

[SÍ, excepcional] LIMITACIONES.md (439 líneas, 13 secciones): lo que NO puede, bugs conocidos con estado, capacidades a medias, números concretos, seguridad completa, clasificación de 53 herramientas externas y qué jamás se integrará. Es el documento que define la cultura del proyecto.

172. Verdad vs marketing

[SÍ] La política es contractual: cada afirmación del README tiene su contrapartida crítica; los 'NO' y 'PARCIAL' abundan; la versión honesta precede a la útil (falsa modestia: cero).

173. Ejemplos ejecutables

[SÍ] 6 en examples/: pool.example.json, pool.mock.json (escenario reproducible), mock_llm_server.py (servidor real para probar todo sin claves), sorl_demo.py, fsm/watch.example.json. Cada uno corre tal cual.

174. Mapa de capacidades

[SÍ] a2s map imprime el mapa directiva->implementación; la tabla del README es su versión estática; cada versión añade su fila (v1.2-v1.6 visibles).

175. Documentación de API interna

[PARCIAL] Docstrings completos en las clases públicas (ProviderPool, FSMEngine, Learner, Telemetry) con semántica de contratos; sin referencia de API generada (sphinx/pdoc: roadmap dev-only).

176. Changelog formal

[PARCIAL] La historia vive en commits semánticos detallados y en los blockquotes de versión del README; sin CHANGELOG.md dedicado con diff de comportamiento por versión (gap menor).

177. Trazabilidad documento-código

[SÍ] Las secciones de LIMITACIONES se citan desde el código (comentarios 'ver §10.3') y los docstrings citan los bugs corregidos: la documentación y el código se refieren mutuamente con anclas estables.

178. Reproducibilidad de las afirmaciones

[SÍ] Cada afirmación cuantitativa de los docs es reproducible con un comando (tests, demos con servidor mock, métricas en estado JSON). Nada de 'hasta 10x más rápido' sin comando adjunto.

179. Onboarding de nuevo desarrollador

[SÍ] En una tarde: 6,5 kLOC stdlib sin framework, arquitectura plana, doctor/map como tour guiado, LIMITACIONES como mapa de minas. La curva de entrada es de las más bajas que puede tener un sistema de agentes.

Categoría 10 (añadida) ? Ética, Legalidad y Fronteras de Diseño

180. Frontera ética como arquitectura

[Sí, rasgo definitorio] El rechazo a ataques a terceros está EN el código (classify_forbidden), no en un documento: el sistema no puede 'decidir' atacar. Redefinir palabras no cambia la conducta (§1.1: decisión permanente).

181. La lección SORD institucionalizada

[Sí] El anti-patrón (usar recursos ajenos sin consentimiento rebautizado como 'agregación') está documentado como tal en `directiva.py` y `LIMITACIONES`, con su equivalente legítimo implementado (SORL). El proyecto recuerda su propio punto de inflexión: la línea existe porque se discutió, no por casualidad.

182. Respeto de términos de servicio ajenos

[Sí] Rate limits auto-impuestos por debajo de los reales (20/min vs 30 de GitHub; rpm por free tier en el pool), Retry-After obedecido siempre, 1 petición mínima en health checks, presupuesto de llamadas por sesión.

183. Licencias de código estudiado

[Sí] Toda ficha registra licencia SPDX de origen antes de asimilar; el conocimiento se cita (fuente+extracto); nunca se copia código a binario propio. Higiene legal de aprendizaje automático, rara de ver.

184. Consentimiento como requisito de recursos

[Sí] Solo recursos del operador (sus claves, su Ollama) o públicos de lectura (GitHub público); el diseño futuro de cómputo ajeno exige consentimiento explícito (BOINC: voluntarios, §12).

185. No disimulo de identidad

[Sí] User-Agent honesto (A2S-*), sin rotación de huellas 'para simular usuarios', jitter declarado como educación de red, no camuflaje. El sistema es identificable en cada petición que hace.

186. Veracidad de autofiguración

[Sí] Se llama 'Agente Autónomo Supremo' pero documenta que no piensa sin LLM, que su heurística empareja palabras, que puede dar falsos cumplidos: el nombre es marca irónica, el contenido es sobrio. La confianza se construye con §1-13.

187. Dual-use forense

[Sí] Las capacidades forenses son defensivas (inventario, hashes, cadena de custodia sobre lo PROPIO); las herramientas ofensivas del estado del arte están clasificadas como no integrables (§9) y el modelo de permisos bloquea su uso desde objetivos.

188. Impacto en recursos compartidos

[Sí] El free-first con cuotas auto-impuestas minimiza la tragedia de los comunes: el sistema deja cuota para otros usuarios de los mismos free tiers (decisión documentada, no obligación legal).

189. Trazabilidad ética de decisiones

[Sí] Cada denegación, cuarentena y degradación queda en ledger/telemetría: se puede auditar que el sistema se comportó según sus principios en cada instante.

190. Antiforense

[NO por diseño] Ninguna capacidad de borrar huellas en sistemas ajenos; la única 'higiene' es el borrado seguro del workspace PROPIO (documentada como excepción legítima).

191. Responsabilidad del operador

[Sí] Los peligros reales se declaran con instrucciones (--public con advertencia, --unsafe 'bajo tu responsabilidad', webhook 'detrás de reverse proxy'): autonomy con caveat emptor explícito, no oculto.

192. Sesgo algorítmico interno

[PARCIAL] Los sesgos del propio aprendizaje se documentan (popularidad por estrellas, win-rate sin decay); la puerta de incompetencia evita que un sesgo temporal se perpetúe bloqueando endpoints por 429s ajenos a su fiabilidad. Sin auditoría formal de equidad (no aplica a dominio).

193. Cumplimiento normativo

[PARCIAL] Herramienta local de operador: sin datos de terceros, sin RGPD directo, sin cookies de tracking. Si se despliega como servicio multiusuario, faltan RGPD/LOS (datos, aviso, borrado): gap de producto, no de diseño actual.

Categoría 11 (añadida) ? Operación, Despliegue y Coste

194. Instalación

[Sí, trivial] pip install -e . (puro) o NADA: el zipapp de ~500 KB corre con solo python3 en el host. Cero pasos de resolución de dependencias: el despliegue más simple posible.

195. Despliegue en vivo (LiveCD)

[Sí] a2s build-live empaqueta un solo archivo ejecutable; --ram opera entero en /dev/shm (nada toca disco): modo respuesta a incidentes con huella cero.

196. Configuración declarativa

[Sí] JSON para pool/watch/fsm con validación estática y mensajes de error accionables; env-vars para secretos; flags para operación puntual. Sin YAML (deliberado: stdlib) ? menos ergonomía, cero dependencias.

197. Provisionamiento de infraestructura

[NO] Sin Terraform/Ansible/K8s propios: el diseño §12 declara el provisionador spot como herramienta del OPERADOR. A²S se despliega, no se autodespliega (elección: el agente no replica infraestructura sin control humano).

198. Upgrades y migraciones

[PARCIAL] Versionado semántico correcto; los formatos persistidos (state.json, fichas, ledger) evolucionan con tolerancia a claves nuevas; sin framework de migraciones: un cambio de esquema mayor requeriría conversor manual (aún no ha hecho falta).

199. Rollback de versión

[PARCIAL] Volver atrás = workspace viejo + zipapp viejo (todo es archivo). Sin compatibilidad garantizada hacia atrás de snapshots futuros no prometida: avisado en README.

200. Modelo de coste por tokens

[Sí, optimizado] Nivel 0 = 0 tokens. Nivel 1 con pool = free-first medido (coste estimado por endpoint en \$), pago solo tras puerta de incompetencia (demostrado: \$0,0023 en las 6 únicas tareas que solo el pago sabía hacer). Fallback heurístico gratis para siempre.

201. Coste de cómputo local

[Sí, mínimo] Proceso Python sin frameworks: RAM de decenas de MB, CPU solo en misiones; SQLite/JSONL: I/O trivial. Puede correr en una Raspberry Pi.

202. Coste de desarrollo/mantenimiento

[PARCIAL] La deuda medida (criterio 16) se estima en 2-3 jornadas de refactor. El coste de reimplementar (PDF, shell, HTTP) ya está pagado y auditado; el de SEGUIR sin linter/CI se paga en cada regresión evitada a mano.

203. Observabilidad en producción

[PARCIAL] JSONL+JSON exportables a cualquier pila (documentado el camino a Grafana); sin exporters nativos Prometheus/OTLP (roadmap declarado §10.3).

204. Runbooks operativos

[NO] Sin manual de incidentes del propio sistema (los mensajes accionables parcialmente lo sustituyen: cada error dice qué revisar). Gap de proceso.

205. Ejecución como servicio (daemon)

[Sí] watch es un daemon real (eventos + webhook + parada por inactividad); dashboard sirve en puerto; supervise es el daemon de misión. Sin systemd units de ejemplo (facilísimo añadir).

206. Multi-instancia

[PARCIAL] Swarm replica procesos con workspaces aislados (el patrón soportado); no hay coordinación entre instancias en la misma máquina (ni lo necesita el modelo single-writer).

207. Presupuesto de coste como parámetro de primer orden

[Sí] El coste es una dimensión de la función de utilidad (weight cost, auto-ajustada), con techo por tier y medición contable: se puede operar con 'gasta solo si es imprescindible' como política explícita.

Categoría 12 (añadida) ? Escalabilidad y Evolución Futura

208. Escalado vertical

[SÍ] El fanout escala con hilos hasta los límites de cuota (el cuello es la cuota ajena, no la CPU local); swarm usa todos los núcleos (procesos).

209. Escalado horizontal distribuido

[NO/por diseño] Sin clúster propio: el roadmap legítimo es BOINC voluntario y spot (diseño §12). La coordinación distribuida se esquila a propósito (la complejidad no compra capacidad real para el dominio).

210. Colas de mensajes

[NO] Sin RabbitMQ/Kafka: el ledger JSONL ES la cola duradera local (append-only consumible). Para un solo operador, correcto; para flota, insuficiente.

211. Escalado de datos

[PARCIAL] SQLite aguanta GBs de episodios; latencias con memoria acotada (deque maxlen); telemetry.jsonl crece sin rotación (gap menor). Sin sharding ni BD externa: por diseño local-first.

212. Límites de concurrencia internos

[SÍ medidos] GIL irrelevante (I/O-bound); 8 hilos por defecto en fanout; ProcessPool con tope por workers; locks cortos sin contención observable (tests de carrera estables).

213. Escalado del pool de proveedores

[SÍ] Añadir endpoints es editar JSON; el scheduler degrada utilidad $O(1)$ por candidato: decenas de endpoints sin costo algorítmico. El límite real es cuántas claves legítimas tengas.

214. Escalado del conocimiento

[PARCIAL] Las fichas top-4 se inyectan por win-rate (ventana acotada al contexto); más allá de ~30 fichas el ranking importa y no hay búsqueda semántica: el corpus crecido necesita embeddings (roadmap).

215. Extensión a nuevos motores de razonamiento

[SÍ] BaseProvider + factory: cualquier motor futuro (Temporal, LangGraph, un modelo local) se conecta implementando 4 métodos; el pool lo orquesta con los demás.

216. Granularidad de checkpoint

[SÍ] Episodio (paso), estrategia (misión), snapshot (sesión): tres granularidades de persistencia alineadas con los tres horizontes de aprendizaje.

217. Evolución de la especificación (versionado de specs)

[PARCIAL] Las specs FSM/watch/pool se validan estáticamente y toleran claves nuevas; sin campo 'version' formal ni migración: si un formato cambia de forma incompatible, fallará la validación con error claro (fallback aceptable).

218. Camino a multiusuario real

[NO] Requiere: RBAC, workspace por usuario, aislamiento de secretos, dashboard auditado. Ninguno bloqueado arquitectónicamente, ninguno empezado: hoy es herramienta de operador.

219. Camino a razonamiento profundo local

[PARCIAL] El conector Ollama ya permite LLM local gratis; el sistema no empaqueta modelos (LiveCD de 500 KB vs GBs de pesos): decisión de producto coherente, con el puente listo cuando el operador lo instale.

Categoría 13 (añadida) ? Comparativa con el Estado del Arte

220. vs Auto-GPT/BabyAGI/AgentGPT

[DIFERENCIAL] A²S es determinista-audio-table (6,5 kLOC leíbles), sin dependencias, con cadena de custodia y honestidad contractual; aquellos ofrecen más plugins comunitarios pero cero auditabilidad profunda y presupuesto de tokens opaco.

221. vs frameworks de agentes (LangChain/LlamaIndex/CrewAI)

[TRADE-OFF] Aquí no hay lock-in de framework ni abstracciones mudables; faltan sus conectores (vectores, tools marketplaces, tracing integrado). A²S eligió stdlib: coste de ecosistema, ganancia de control.

222. vs orquestadores duraderos (Temporal/Airflow)

[TRADE-OFF] El ledger es workflow duradero local (append-only, reanudable) sin servidor de orquestación; faltan timers distribuidos, retries centralizados y flota de workers. Para un operador: suficiente; para una org: no.

223. vs gateways LLM (LiteLLM/Portkey)

[DIFERENCIAL] El pool SORL es un gateway CON aprendizaje propio (rpm observado, aptitud medida, pesos adaptativos, puerta de incompetencia): los gateways enrutan por regla; SORL aprende del resultado.

224. vs sistemas anti-límites (el 'SORD' del mercado gris)

[OPUESTO POR PRINCIPIO] Existen herramientas de rotación de IPs/proxies para sortear rate limits: A²S documenta ese patrón como anti-patrón y hace lo contrario (cuotas bajo el límite + Retry-After obedecido). La comparativa es ética, no técnica.

225. vs suites forenses (Plaso/Sleuth/Volatility)

[COMPLEMENTARIO] A²S no las sustituye: las puentea (v1.3) con lista blanca y confina su salida; aporta lo que ellas no tienen: agente que persigue el objetivo y firma la cadena.

226. vs cómputo voluntario (BOINC/Folding)

[DISEÑADO, NO IMPLEMENTADO] §12 especifica el camino consentido (tareas embolsadas, resultados firmados HMAC, verificación por muestreo): las piezas criptográficas ya existen en el repo; falta el servidor de colas público.

227. vs evaluación de agentes (AgentBench/SWE-bench)

[NO] Sin benchmark externo formal de capacidades del agente (las verificaciones son internas y demos reproducibles). Integrar un benchmark público daría métrica comparativa dura: mejora recomendada.

228. vs memo-ria de agentes (vector DBs: Chroma/Qdrant)

[GAP DECLARADO] Sin embeddings ni búsqueda semántica: la memoria es episódica SQL+estratégica. Para corpus pequeños funciona; para bibliotecas de conocimiento grandes se queda corto (roadmap nº1 de capacidades).

229. Posición honesta en el mapa

[SÍ] A²S ocupa un nicho real y poco poblado: agente autónomo AUDITABLE, barato por arquitectura (dos niveles), con aprendizaje medido y fronteras éticas ejecutables. No compite en plugins, ni en escala, ni en profundidad LLM: compite en confianza y coste total.

Categoría 14 (añadida) ? Síntesis, Riesgos y Veredicto

230. Puntuación C1 Arquitectura

4,0/5 ? Sana, medida, explicable; le faltan los refactors de los 5 hotspots y desdoblar cli.py.

231. Puntuación C2 Funcionalidad

3,5/5 ? El núcleo prometido está completo y verificado en vivo; faltan i18n, notificaciones salientes y multiusuario.

232. Puntuación C3 UI/UX

2,5/5 ? Excelente CLI para operadores; dashboard básico; sin accesibilidad formal ni personalización.

233. Puntuación C4 Capacidad analítica

3,5/5 ? Escalera de recuperación y optimización mediana excelentes; el razonamiento profundo depende del LLM externo (y lo dice).

234. Puntuación C5 Crecimiento autónomo

3,0/5 ? Aprende de fallos, cuotas, aptitud y GitHub con auditoría; sin unlearning de fichas ni búsqueda semántica.

235. Puntuación C6 Seguridad

3,5/5 ? Modelo de permisos ejecutable, sandbox por capas, cripto verificable, cero supply-chain; dashboard sin auditar y webhook sin auth (documentados).

236. Puntuación C7 Fiabilidad

3,5/5 ? Degradación en cascada demostrada, checkpoints, circuitos; sin alertamiento ni backups automáticos.

237. Puntuación C8 Código y pruebas

3,5/5 ? 139 tests disciplinados con regresión por bug; sin cobertura medida, CI ni fuzzing.

238. Puntuación C9 Documentación

5,0/5 ? LIMITACIONES.md es la mejor documentación de límites que se puede encontrar en un proyecto de este tipo; ejemplos ejecutables de todo.

239. Puntuación C10 Ética

5,0/5 ? Fronteras en código, no en papel; licencias respetadas en el aprendizaje; anti-patrón SORD documentado con su alternativa legítima implementada.

240. Puntuación C11-C13 Operación/Escalabilidad/Comparativa

C11 3,5/5 (despliegue trivial, coste optimizado, sin provisionamiento) · C12 3,0/5 (local por diseño, camino distribuido consentido solo diseñado) · C13 3,5/5 (nicho propio: auditabilidad+coste).

241. Fortaleza nº1

La confianza verificable: cada afirmación de este informe se reproduce con un comando; cada decisión del sistema deja huella criptográfica; cada límite está escrito donde el marketing habría puesto una feature.

242. Fortaleza nº2

El coste como dimensión arquitectónica: dos niveles (determinista gratis / agente solo si hace falta) + pool free-first con puerta de incompetencia = capacidad real con presupuesto ~0 (demostrado con \$0,0023).

243. Fortaleza nº3

El aprendizaje auditado: rpm aprendido, pesos adaptativos, aptitud medida por kind y fichas de conocimiento con licencia: el sistema mejora Y puede demostrar por qué.

244. Fortaleza nº4

Cero dependencias: sin CVEs, sin lockfile, LiveCD de 500 KB, auditabilidad total en una tarde de lectura.

245. Fortaleza nº5

La honestidad como ingeniería: LIMITACIONES §1-13 convierte el 'qué no puede' en contrato; los bugs se celebran con test de regresión.

246. Debilidad nº1

El razonamiento heurístico da falsos positivos de éxito sin verificador de misión (§2 nº9): el usuario novato puede creerse un 'CUMPLIDO' flojo. Mitigación documentada, no resuelta.

247. Debilidad nº2

Hotspots de complejidad (CC hasta 33) y cli.py creciendo: deuda medida que dificulta contribuyentes externos.

248. Debilidad nº3

Ecosistema ausente: sin memoria vectorial, sin conectores, sin CI, sin multiusuario: para equipos y corpus grandes hoy se queda corto.

249. Riesgo principal

Confundir la herramienta de operador con un producto: exponer dashboard/webhook sin endurecer (auth, TLS, auditoría web) o desplegarlo multiusuario sin RGPD sería usarlo contra sus propias instrucciones.

250. VEREDICTO

A²S v1.6.0 es lo que promete: un agente autónomo de operador, auditable y barato, con tres bucles de mejora legítimos (orquestración SORL, optimización medida, enriquecimiento CE) y un nivel determinista que cubre lo predecible a coste cero. No es, ni finge ser, un cerebro: es una máquina de perseguir objetivos con memoria, frenos y recibos. Global ponderado: 3,7/5 ? con la mejor relación transparencia/capacidad del nicho.

Roadmap priorizado (derivado de este análisis)

- 1. CI mínima: GitHub Actions con los 139 tests (6 líneas, cero deps).
- 2. Refactor de los 5 hotspots CC>19 con tests-first (shell, execute_dag, evolve_step, _handler, execute_step).
- 3. Memoria semántica local opcional (índice invertido BM25 stdlib antes que embeddings externos).
- 4. Alertamiento saliente: webhook/email cuando el pool degrade o el watcher escale.
- 5. Unlearning real: caducidad y poda de fichas perdedoras; decay de win-rates.
- 6. search/code en el CE (estudiar fragmentos, no solo READMEs) con los mismos presupuestos y permisos.
- 7. Auditoría web del dashboard (CSRF/headers/ARIA) antes de cualquier exposición.
- 8. a2s learn que EDITE la especificación FSM a partir de los escalados (cerrar el ciclo nivel 1->nivel 0).
- 9. Rotación/compresión de telemetry.jsonl y CHANGELOG.md formal.
- 10. Provisionador spot del §12 como herramienta del operador (no del agente).

Generado con tools/gen_informe.py (motor PDF en stdlib puro, sin dependencias). Todas las afirmaciones son reproducibles con los comandos citados en README.md y LIMITACIONES.md.